

Operating Systems — ★★★Exam Answers (Chapters 1–5)

Based on: *Operating System Concepts, 10th Edition* — Silberschatz, Galvin & Gagne

Mahbub Sir's Part | Full Exam-Style Answers

Note: All answers are written in exam format. ★★★ = Most repeated questions — prioritize these.

CHAPTER 1: INTRODUCTION

Q1. What are the three main purposes of an operating system?

★★★ [17-18 FINAL] [18-19 MID] [18-19 FINAL] [20-21 FINAL] — *Book Exercise 1.1*

Introduction

An **Operating System (OS)** is a program that acts as an intermediary between a user of a computer and the computer hardware. According to Silberschatz, the OS has three primary purposes.

Three Main Purposes

1. Resource Allocator

- The OS manages all hardware resources (CPU, memory, I/O devices, files).
- When multiple programs or users compete for the same resources simultaneously, the OS must decide **who gets what, when, and for how long**.
- Goal: Operate the computer system **efficiently** and fairly.

Example: The CPU scheduler decides which process gets the CPU next.

2. Control Program

- The OS acts as a **control program** that manages the execution of user programs to prevent errors and improper use of the computer.
- It protects the system from malicious or erroneous programs that could crash the entire system.

Example: A program cannot directly access another program's memory region — the OS enforces memory protection.

3. Convenience Provider (User Interface)

- The OS makes the computer **convenient to use** by providing an abstraction over complex hardware.
- Without an OS, users would have to manage hardware directly in binary/machine language.
- It provides services like: file management, process execution, I/O operations, etc.

Example: A user can simply call `open("file.txt")` without knowing the actual disk sector address.

Summary Table

Purpose	Goal	Example
Resource Allocator	Efficient resource management	CPU scheduling, memory allocation
Control Program	Error prevention and protection	Memory protection, access control
Convenience Provider	Ease of use	File system, GUI, system calls

Q2. When is it appropriate for the OS to “waste” resources? Why is such a system not really wasteful?

★★★ [17-18 FINAL] [18-19 MID] [18-19 FINAL] — *Book Exercise 1.2*

Introduction

While the OS is generally designed to **maximize efficiency** of hardware resources, there are scenarios where it is appropriate to deliberately sacrifice efficiency. This may seem wasteful, but from a broader perspective, it is not.

When It Is Appropriate to “Waste” Resources

1. Single-User Interactive Systems

- When a computer is used by a **single user** (e.g., a personal laptop), dedicating maximum resources to that user’s experience is more important than raw hardware efficiency.
- The OS may leave CPU idle rather than running background tasks that would slow down the user’s application.

2. Maximizing User Convenience and Response Time

- The OS may allocate extra memory or CPU cycles to keep a **GUI responsive** even if that means the CPU is not 100% utilized.

- In time-sharing systems, the OS gives each user a small time quantum — this increases context-switch overhead, but improves **perceived responsiveness**.

3. Real-Time Systems

- In real-time systems, the OS must meet **deadlines**, even if it means keeping resources reserved/idle to be instantly available.
- Wasting some CPU time guarantees a process can execute within its time constraint.

4. Development / Testing Environments

- A system may allocate extra resources for debugging, logging, or monitoring — activities that are “wasteful” in production but necessary during development.

Why Such a System Is Not Really Wasteful

The key insight is:

The goal of a computer system is to solve problems for users — not just to efficiently use hardware.

- If “wasting” CPU or memory leads to a **better user experience**, faster response times, or more reliable service, then the overall **value delivered** is maximized.
- Resources that seem idle (e.g., reserved memory) may be enabling **faster response** when needed.
- Efficiency is only meaningful when it serves the **goals of the users and the organization**.

Example: A web server that keeps a pool of pre-allocated threads “wastes” memory, but it eliminates the delay of creating a new thread for each incoming request — improving overall throughput and response time.

CHAPTER 2: OPERATING-SYSTEM STRUCTURES

Q3. What is the purpose of system calls?

★★★ [17-18 FINAL] [18-19 MID] [18-19 FINAL] [19-20 FINAL] [20-21 FINAL] —
Book Exercise 2.1

Introduction

A **system call** provides the interface between a running program (user process) and the operating system. It is the mechanism through which a user-mode process requests a service from the OS kernel.

Purpose of System Calls

System calls serve the following essential purposes:

1. Provide Controlled Access to Hardware

- User programs cannot directly access hardware (disk, memory, network) for security reasons.
- System calls provide a **safe, controlled gateway** into kernel mode where hardware access is permitted.

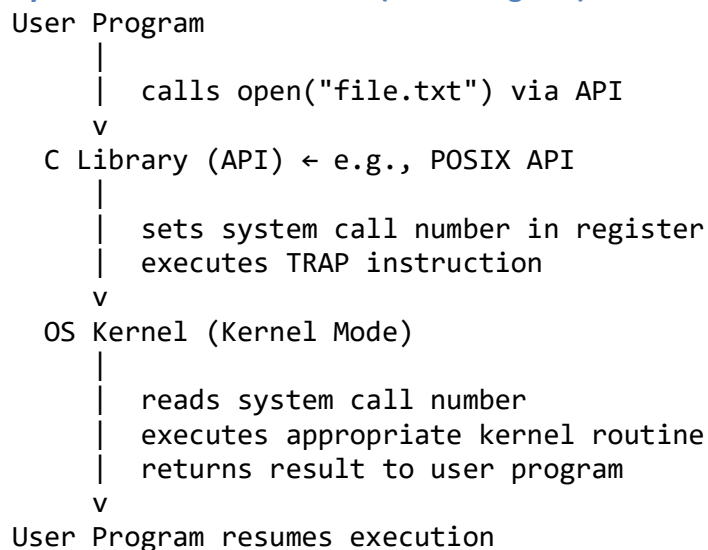
2. Provide OS Services to User Programs

System calls expose OS services such as: - **File operations**: `open()`, `read()`, `write()`, `close()` - **Process control**: `fork()`, `exec()`, `exit()`, `wait()` - **Memory management**: `mmap()`, `brk()` - **Communication**: `pipe()`, `socket()`, `send()`, `recv()` - **Device management**: `ioctl()`

3. Mode Switching: User Mode → Kernel Mode

- When a system call is invoked, the CPU switches from **user mode to kernel mode** (via a software interrupt/trap).
- After the OS executes the requested service, control returns to the user program in user mode.

System Call Execution Flow (ASCII Diagram)



4. Protection and Isolation

- System calls enforce the **dual-mode operation** of the OS.
- They prevent user programs from interfering with each other or corrupting OS data structures.

Real-World Connection (Linux/UNIX)

In Linux, system calls are invoked via the `syscall` instruction. The `strace` command can be used to trace all system calls made by a process.

Example: Running `strace ls` shows dozens of system calls like `open()`, `read()`, `write()`, `close()` that the simple `ls` command makes internally.

Q4. Describe three general methods for passing parameters to the operating system.

★★★ [17-18 MID] [18-19 FINAL] [20-21 FINAL] — *Book Exercise 2.10*

Introduction

When a process makes a **system call**, it often needs to pass parameters to the OS (e.g., the name of a file to open, or the address of a memory buffer). The OS and the calling program must agree on the parameter-passing mechanism.

Three Methods of Parameter Passing

Method 1: Using Registers

- Parameters are placed directly into **CPU registers** before the system call.
- The OS reads the registers to retrieve the parameters.

Advantages: - Very fast — no memory access needed.

Disadvantages: - Limited number of registers → cannot pass many parameters. - Not suitable when parameter data is large.

Register R0 ← system call number

Register R1 ← parameter 1

Register R2 ← parameter 2

TRAP

Method 2: Using a Block (Table) in Memory

- Parameters are stored in a **contiguous block of memory**.
- The **address of the block** is passed to the OS in a register.
- Used when the number of parameters exceeds the number of available registers.

Advantages: - Can pass any number of parameters of any size. - Used by Linux for this reason.

Disadvantages: - Slightly slower due to additional memory access.

Memory Block:

[param1 | param2 | param3 | param4]

^

|

Register R1 ← address of block

TRAP (OS reads block via address in R1)

Method 3: Using the Stack (Program Stack)

- Parameters are **pushed onto the process's stack** by the user program.
- The OS **pops** parameters from the stack.

Advantages: - Does not limit the number of parameters. - Natural for programs that already use a call stack.

Disadvantages: - Slightly more overhead due to stack push/pop operations.

Stack:

[param3] ← top

[param2]

[param1]

TRAP (OS pops parameters from stack)

Comparison Table

Method	Speed	# of Params	Memory Used
Registers	Fastest	Limited (few)	None
Memory Block/Table	Medium	Unlimited	Yes (block)
Stack	Medium	Unlimited	Yes (stack)

Note: Methods 2 and 3 do not limit the number or length of parameters. Linux primarily uses a combination of registers and memory block.

Q5. Compare and contrast monolithic, layered, microkernel, modular, and hybrid OS designs.

★★★ [19-20 FINAL] [20-21 FINAL]

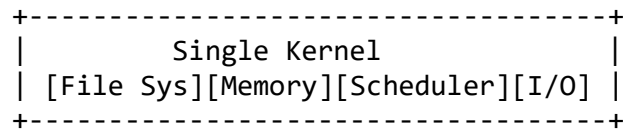
Introduction

The **structure of an operating system** significantly affects its performance, maintainability, and extensibility. Silberschatz describes five major OS design approaches.

1. Monolithic Structure (Simple / No Structure)

- All OS functionality is combined into a **single large kernel** that runs entirely in kernel mode.
- There is no well-defined boundary between components.

Example: Early UNIX, MS-DOS.



Advantages: - Excellent performance (direct function calls, no message passing). - Simple to implement initially.

Disadvantages: - Difficult to maintain, debug, and extend. - A bug in one part can crash the entire system.

2. Layered Approach

- The OS is divided into **N layers** (levels).
- Layer 0 = hardware; Layer N = user interface.
- Each layer can only use services of the layer **immediately below** it.

Example: THE Operating System (Dijkstra).

```
Layer N   : User Interface
Layer N-1 : I/O Management
...
Layer 1   : Memory Management
Layer 0   : Hardware
```

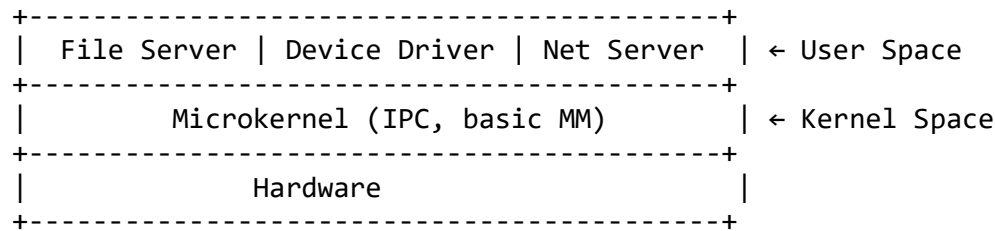
Advantages: - Simplicity of construction and debugging (debug layer by layer). - Modularity and abstraction.

Disadvantages: - Defining the layers is difficult (circular dependencies). - Poor performance due to multiple layer traversals for a single operation. - Overly rigid hierarchy.

3. Microkernel Structure

- Only **essential functions** (inter-process communication, basic scheduling, basic memory management) are kept in the kernel.
- All other services (file systems, device drivers, networking) run as **user-level processes (servers)**.
- Components communicate via **message passing**.

Example: Mach, QNX, macOS (partially), Windows NT (originally).



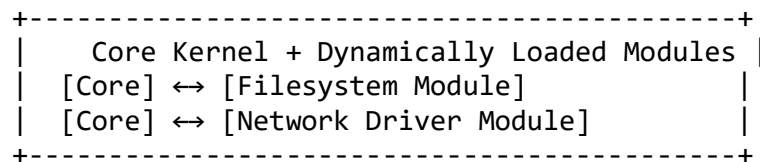
Advantages: - Easier to extend — add a new service without modifying the kernel. - More reliable and secure — a crashed server does not crash the kernel. - Easier to port to new hardware.

Disadvantages: - **Performance overhead** due to message passing between user-space servers and kernel. - More complex communication mechanisms.

4. Modules (Loadable Kernel Modules — LKM)

- The kernel has a **core set of components**, and additional modules can be dynamically loaded/unloaded at runtime.
- Each module communicates with others through well-defined interfaces.
- Uses **object-oriented** principles.

Example: Linux (most modern Linux device drivers are LKMs), Solaris.



Advantages: - More flexible than layered approach (any module can call any other). - More efficient than microkernel (no message passing overhead). - Modules can be loaded/unloaded without rebooting.

Disadvantages: - Modules run in kernel space — a faulty module can crash the kernel.

5. Hybrid Structure

- Most modern OSes use a **combination** of approaches.
- They may have a microkernel-like design but allow some services to run in kernel space for performance.

Examples: - **Linux:** Monolithic kernel + loadable modules. - **macOS:** Hybrid — Mach microkernel + BSD UNIX layers + I/O Kit (OOP modules). - **Windows:** Hybrid — layered design + monolithic kernel-mode components.

Comprehensive Comparison Table

Approach	Kernel Size	Performance	Reliability	Extensibility	Example
Monolithic	Large	Excellent	Low	Poor	Early UNIX
Layered	Medium	Poor	Medium	Medium	THE OS
Microkernel	Very Small	Poor	High	High	Mach, QNX
Modular	Medium	Good	Medium	High	Linux
Hybrid	Variable	Good	Good	Good	macOS, Windows

CHAPTER 3: PROCESSES

Q6. Define Process. What are the different parts of a process?

★★★ [11-12 FINAL] [16-17 FINAL] [19-20 FINAL]

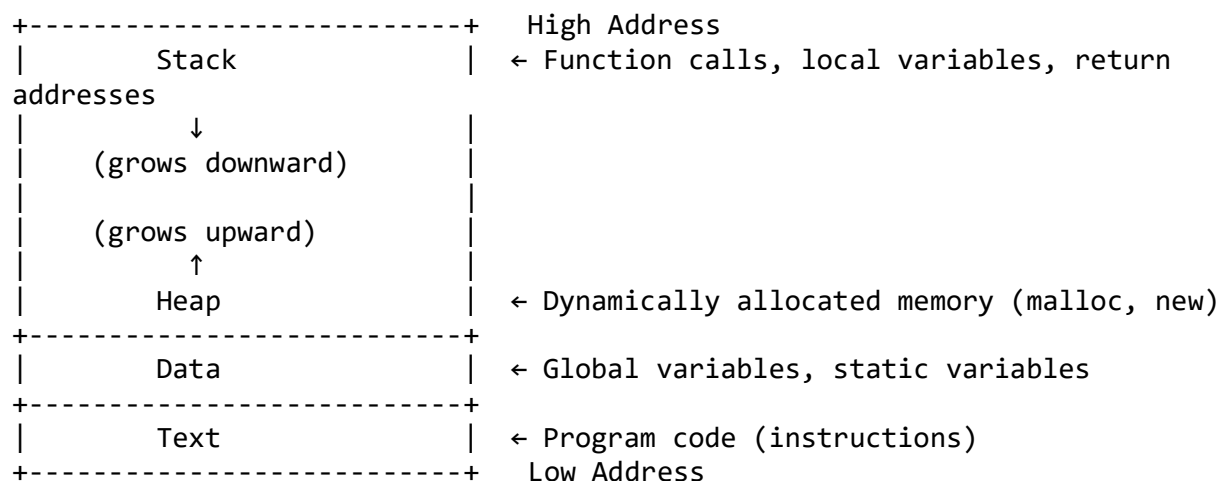
Definition

A **process** is a program in execution. It is the **active entity** — while a program is a **passive entity** (a file on disk), a process is a dynamic entity with associated resources.

Formally: A process is a unit of work in a system. A program becomes a process when it is loaded into memory and begins execution.

Parts / Components of a Process

A process in memory is divided into the following sections:



1. Text Section (Code Segment)

- Contains the **executable code** (machine instructions) of the program.
- Usually **read-only** to prevent accidental modification.

2. Data Section

- Contains **global variables** and **static variables**.
- Initialized data segment: variables with initial values.
- Uninitialized data segment (BSS): variables without initial values.

3. Heap

- Memory that is **dynamically allocated** at runtime.
- Grows upward toward higher memory addresses.
- Managed via malloc(), calloc(), new and freed via free(), delete.

4. Stack

- Contains **temporary data** for function execution:
 - Function parameters
 - Return addresses
 - Local variables
- Grows **downward** toward lower memory addresses.
- Each function call creates a new **stack frame**; each return pops a frame.

Additional Process Information

Besides the memory layout, a process is also represented by its **Process Control Block (PCB)**, which stores: - Process state - Program counter - CPU registers - Memory management information - I/O status - Scheduling information

Q7. What are different process states? Draw the state diagram for a process.

★★★ [11-12 FINAL] [16-17 FINAL] [18-19 FINAL]

Introduction

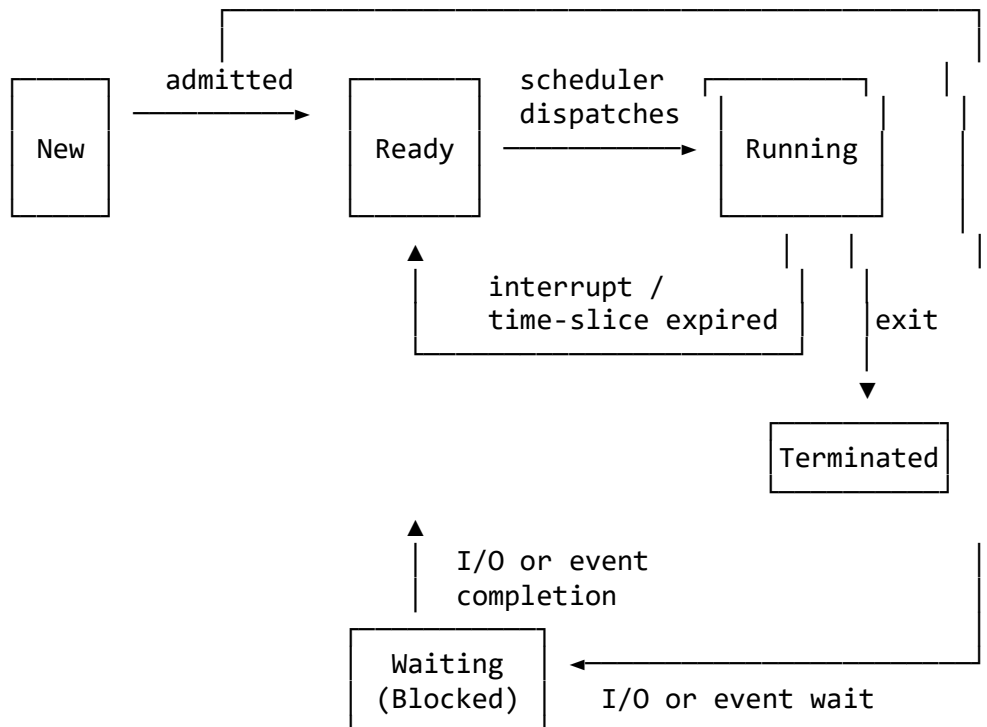
As a process executes, it transitions through a series of **states** that define its current activity. The OS tracks the state of every process.

Five Process States

State	Description
New	The process is being created
Ready	The process is waiting to be assigned to the

State	Description
Running	CPU
Waiting (Blocked)	Instructions are being executed on the CPU
Terminated	The process is waiting for an event (e.g., I/O completion)
	The process has finished execution

Process State Diagram (ASCII)



Detailed State Transitions

1. **New → Ready:** Process is admitted into the system; OS creates the PCB and loads it into the ready queue.
2. **Ready → Running:** The **scheduler (dispatcher)** selects the process and assigns it to the CPU.
3. **Running → Ready:** The process is **preempted** (e.g., time quantum expired) or a higher-priority process arrives.
4. **Running → Waiting:** The process issues an I/O request or waits for an event (e.g., read() from disk). CPU is released.
5. **Waiting → Ready:** The I/O operation completes or the awaited event occurs; the process is placed back in the ready queue.
6. **Running → Terminated:** The process finishes execution (exit()) or is killed.

Key Points

- At any instant, **only one process per CPU core** can be in the **Running** state.
 - Multiple processes can be **Ready** or **Waiting** simultaneously.
 - The OS maintains separate **queues** for Ready and Waiting processes.
-

Q8. Differentiate between Shared-Memory systems and Message-Passing systems.

★★★ [16-17 FINAL]

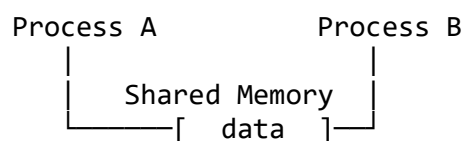
Introduction

Inter-Process Communication (IPC) allows cooperating processes to exchange information and synchronize their actions. The two fundamental models of IPC are **Shared Memory** and **Message Passing**.

Shared-Memory Systems

In a shared-memory system, a **region of memory** is shared between two or more processes. Processes can communicate by reading and writing to this shared region.

- The OS creates the shared memory region.
- After establishment, the OS is **not involved** in further communication — processes access shared memory directly.
- Processes must **synchronize** access (e.g., using semaphores or mutexes) to avoid race conditions.



Classic Example: Producer-Consumer Problem. - The producer writes data into a shared buffer. - The consumer reads data from the shared buffer.

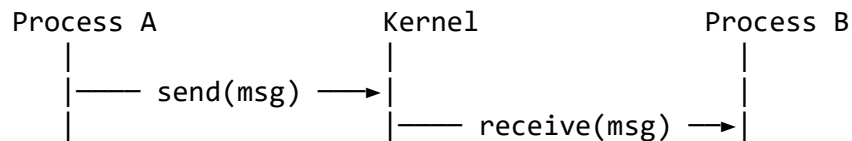
Advantages: - Extremely **fast** — communication occurs at memory speed. - No kernel involvement after setup (no system call overhead per message). - Suitable for **large amounts of data**.

Disadvantages: - **Synchronization** is the programmer's responsibility — complex and error-prone. - Only works efficiently for processes on the **same machine**. - Can lead to **race conditions** if not carefully synchronized.

Message-Passing Systems

In a message-passing system, processes communicate by **sending and receiving messages** via the OS kernel. No shared memory region is needed.

- Each communication requires a **system call** (send/receive).
- The OS acts as an **intermediary**, transferring messages through a kernel buffer.



Primitives: - send(destination, message) - receive(source, message)

Communication Links: - **Direct** or **Indirect** (via mailboxes/ports) - **Synchronous (Blocking)** or **Asynchronous (Non-blocking)**

Advantages: - **Easier to implement** — no synchronization burden on the programmer. - Works well in **distributed systems** (processes on different machines). - Provides **natural synchronization**.

Disadvantages: - **Slower** than shared memory — every message involves kernel intervention (system calls, data copying). - Not efficient for large volumes of data transfer.

Comparison Table

Feature	Shared Memory	Message Passing
Speed	Very fast	Slower
Data Transfer	Direct (via RAM)	Via kernel buffer
Synchronization	Manual (programmer's job)	Built-in
Ease of Use	Difficult	Easier
Suitability	Same machine	Same or different machines
Distributed Systems	Not suitable	Very suitable
Kernel Involvement	Only at setup	Every send/receive
Example	Producer-Consumer buffer	Pipes, Sockets, MPI

CHAPTER 4: THREADS & CONCURRENCY

Q9. Define thread. Identify the basic components of a thread, and contrast threads and processes.

★★★[12-13 FINAL] [19-20 FINAL] [20-21 FINAL]

Definition of a Thread

A **thread** (also called a **lightweight process**) is the basic unit of CPU utilization. It represents a **single sequence of execution** within a process.

A process can have **multiple threads**, all sharing the same process resources but executing independently.

Basic Components of a Thread

Each thread has its own:

Component	Description
Thread ID	Unique identifier for the thread
Program Counter (PC)	Tracks the address of the next instruction to execute
Register Set	CPU registers used by the thread during execution
Stack	Stores local variables and function call information

Thread Memory Layout in a Multi-threaded Process

Thread 1 [Stack]	Thread 2 [Stack]	← Each thread has its own stack
Shared Heap		← Shared among all threads
Shared Data		← Global variables shared
Shared Code		← Code (text) shared

Threads share: Code section, data section, heap, open files, OS resources. **Threads have their own:** Stack, program counter, registers, thread ID.

Contrast: Thread vs. Process

Feature	Process	Thread
---------	---------	--------

Feature	Process	Thread
Definition	Program in execution	Unit of execution within a process
Resources	Has its own code, data, heap, stack, files	Shares code, data, heap, files with other threads
Creation overhead	High (new address space, PCB, resources)	Low (shares existing process resources)
Context switch cost	High	Low
Communication	Requires IPC (shared memory / message passing)	Can communicate via shared memory directly
Fault isolation	Crash of one process does not affect others	Crash of one thread can crash the whole process
Concurrency	Multiple processes can run concurrently	Multiple threads within one process can run concurrently

Real-World Connection

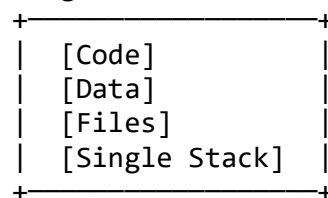
In Linux, both processes and threads are represented by `task_struct`. Threads are created with the `clone()` system call with flags specifying which resources to share.

Q10. Differentiate between single-threaded and multi-threaded process. Describe the major benefits and significant challenges of multithreaded processes.

★★★ [16-17 FINAL] [19-20 FINAL] [20-21 FINAL]

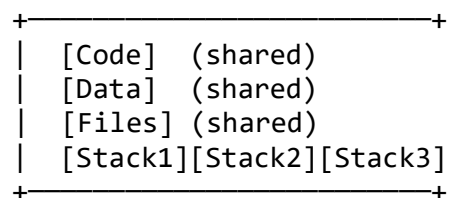
Single-Threaded vs. Multi-Threaded Process

Single-Threaded Process:



↓
One thread of execution

Multi-Threaded Process:



↓ ↓ ↓
Three concurrent threads of execution

Single-Threaded Process: - Only one thread of execution exists. - The process executes instructions **sequentially**. - One blocking operation (e.g., I/O) stalls the entire process.

Multi-Threaded Process: - Multiple threads exist within the same process. - Threads execute **concurrently** (or in parallel on multi-core systems). - One thread can block without stalling others.

Benefits of Multithreading

1. Responsiveness

- A multithreaded application remains responsive to the user even when part of it is blocked.
- *Example: A web browser can continue rendering a page (one thread) while downloading a file (another thread).*

2. Resource Sharing

- Threads share the memory and resources of the process **by default**.
- Allows several threads of activity within the same address space without IPC overhead.
- More efficient than creating multiple processes.

3. Economy (Efficiency)

- Creating and managing threads is **cheaper** than creating processes.
- Context switching between threads is faster than between processes.
- Thread creation is approximately **30× faster** than process creation in Solaris.

4. Scalability / Parallelism

- On **multi-core or multiprocessor** systems, threads can run truly **in parallel** on different cores.
 - A single-threaded process can only use one core, wasting the rest.
 - *Example: A matrix multiplication program can split the work across 8 threads on an 8-core processor.*
-

Challenges of Multithreaded Design

1. Dividing Activities (Identifying Parallelism)

- Deciding which parts of the program can run concurrently is complex.
- Not all tasks are easily parallelizable (Amdahl's Law limits speedup).

2. Data Splitting

- Data must be divided appropriately across threads without causing conflicts or imbalances.

3. Data Dependency

- When one task depends on the result of another, the tasks cannot run simultaneously.

- The programmer must ensure proper synchronization.

4. Testing and Debugging

- Multithreaded programs are significantly **harder to test and debug**.
- Bugs like **race conditions** and **deadlocks** are non-deterministic and hard to reproduce.

5. Synchronization Issues

- Shared data accessed by multiple threads can lead to **race conditions** and **data inconsistency**.
- Requires synchronization mechanisms (mutexes, semaphores), which add complexity and can cause deadlocks.

6. Thread Safety

- Libraries and functions used by threads must be **thread-safe** (reentrant).

Summary Table

Aspect	Single-Threaded	Multi-Threaded
Execution	Sequential	Concurrent/Parallel
Responsiveness	Low (blocking stalls all)	High
Resource use	Wasteful on multi-core	Efficient
Complexity	Simple	Complex (synchronization needed)
Performance	Limited	Higher on multi-core

CHAPTER 5: CPU SCHEDULING

Q11. Explain the difference between preemptive and nonpreemptive scheduling.

★★★ [16-17 FINAL] [17-18 FINAL] [19-20 FINAL] [20-21 FINAL] — *Book Exercise 5.2*

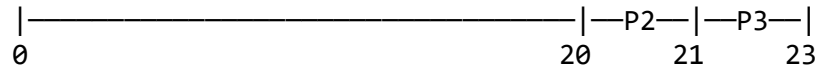
Introduction

CPU scheduling decides which process in the ready queue gets the CPU next. Scheduling decisions happen under different circumstances — and the policy regarding **whether the CPU can be taken away** from a running process defines whether the algorithm is preemptive or nonpreemptive.

Nonpreemptive (Cooperative) Scheduling

- Once the CPU is allocated to a process, the process **keeps the CPU until it voluntarily releases it**.
- The CPU is released only when:
 1. The process **terminates**.
 2. The process switches to the **waiting state** (e.g., for I/O).

P1 runs until done



Advantages: - Simple to implement. - No risk of a process being interrupted mid-operation. - No inconsistency issues with shared data (no preemption mid-critical section).

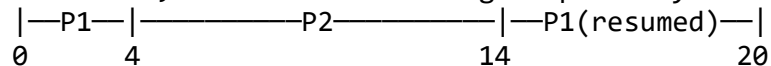
Disadvantages: - A CPU-bound process can monopolize the CPU. - Poor responsiveness and **high waiting times** for other processes. - **Not suitable for real-time or interactive systems**.

Example: Windows 3.1 used cooperative multitasking.

Preemptive Scheduling

- The OS can **forcefully take the CPU away** from a running process and give it to another process.
- Preemption occurs when:
 1. A **higher-priority process** arrives in the ready queue.
 2. A process's **time quantum expires** (in Round Robin).
 3. An interrupt occurs.

P1 starts, P2 arrives with higher priority at $t=4 \rightarrow$ P1 preempted



Advantages: - **Better response time** — high-priority or time-critical processes can get CPU quickly. - **Fairer CPU sharing** among processes. - Essential for real-time and interactive systems.

Disadvantages: - **More complex** to implement. - Requires careful handling of **shared data** (a process preempted mid-update can leave shared data inconsistent — needs synchronization). - **Higher context-switch overhead**.

Example: Linux, Windows (from NT onwards), and macOS all use preemptive scheduling.

Comparison Table

Feature	Nonpreemptive	Preemptive
CPU release	Voluntary (by process)	Forced (by OS)

Feature	Nonpreemptive	Preemptive
Response time	Poor	Good
Fairness	Poor	Better
Implementation	Simple	Complex
Overhead	Low	Higher (context switches)
Use case	Simple/batch systems	Interactive, real-time systems
Example algorithms	FCFS, Non-preemptive SJF	RR, Preemptive SJF (SRTF), Preemptive Priority

Q12. What criteria affect the scheduler's performance? Which ones should be maximized and minimized?

★★★ [11-12 FINAL] [12-13 FINAL] [13-14 FINAL]

Introduction

The performance of a CPU scheduling algorithm is evaluated by several **criteria**. These criteria help compare and select the most appropriate scheduling algorithm for a given system.

Scheduling Criteria

Criteria to MAXIMIZE ↑

Criterion	Definition	Goal
CPU Utilization	Percentage of time the CPU is busy doing useful work	Keep CPU as busy as possible (ideally 40–90%)
Throughput	Number of processes completed per unit time	Maximize processes completing per second/minute

Criteria to MINIMIZE ↓

Criterion	Definition	Goal
Turnaround Time	Total time from process submission to completion (waiting + execution + I/O)	Minimize total elapsed time
Waiting Time	Total time a process spends in the ready queue waiting for CPU	Minimize idle wait
Response Time	Time from submission of a	Minimize time to first output

Criterion	Definition	Goal
	request to the first response (important for interactive systems)	

Formulas

Turnaround Time = Completion Time - Arrival Time
 Waiting Time = Turnaround Time - Burst Time
 Response Time = Time of First CPU Execution - Arrival Time

Typical Optimization Goals

System Type	Priority Criterion
Batch systems	Maximize throughput, minimize turnaround time
Interactive systems	Minimize response time
Real-time systems	Meet deadlines (minimum response time)

Key Observations

- **CPU Utilization** and **Throughput** should be **maximized**.
- **Turnaround Time**, **Waiting Time**, and **Response Time** should be **minimized**.
- In practice, optimizing all criteria simultaneously is impossible — there are **trade-offs** between them (e.g., minimizing average waiting time might increase response time variance).
- Schedulers often try to minimize **average waiting time** as it is the most commonly compared metric.

Q13. CPU Scheduling — FCFS, SJF, Non-Preemptive Priority & RR with Gantt Charts

★★★ [12-13 FINAL] [13-14 FINAL] [17-18 FINAL] [18-19 FINAL] [19-20 FINAL]
 — Book Exercise 5.4, 5.17

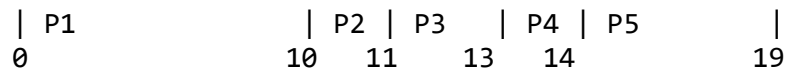
Given Data (Book Exercise 5.4 — All Arrive at Time 0, RR Quantum = 1)

Process	Burst Time	Priority (lower = higher)
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

Algorithm 1: FCFS (First Come, First Served)

Rule: Processes are served in order of arrival. Since all arrive at $t=0$, serve in order P1, P2, P3, P4, P5.

Gantt Chart:



Calculations:

Process	Completion	Turnaround (CT-AT)	Waiting (TAT-Burst)
P1	10	10	0
P2	11	11	10
P3	13	13	11
P4	14	14	13
P5	19	19	14

- **Average Waiting Time** = $(0 + 10 + 11 + 13 + 14) / 5 = 48 / 5 = 9.6 \text{ ms}$
- **Average Turnaround Time** = $(10 + 11 + 13 + 14 + 19) / 5 = 67 / 5 = 13.4 \text{ ms}$

Algorithm 2: SJF (Shortest Job First — Non-Preemptive)

Rule: Select the process with the **shortest burst time**. All arrive at $t=0$. Sorted order by burst: P2(1), P4(1), P3(2), P5(5), P1(10). (*Tie between P2 and P4 resolved by original order.*)

Gantt Chart:



Calculations:

Process	Completion	Turnaround	Waiting
P2	1	1	0
P4	2	2	1
P3	4	4	2
P5	9	9	4
P1	19	19	9

- **Average Waiting Time** = $(0 + 1 + 2 + 4 + 9) / 5 = 16 / 5 = 3.2 \text{ ms} \leftarrow \text{Minimum!}$
- **Average Turnaround Time** = $(1 + 2 + 4 + 9 + 19) / 5 = 35 / 5 = 7.0 \text{ ms}$

SJF gives the minimum average waiting time among all non-preemptive algorithms.

Algorithm 3: Non-Preemptive Priority Scheduling

Rule: Select the process with the **highest priority** (lowest priority number). Sorted: P2(priority 1) → P5(priority 2) → P1(priority 3) → P3(priority 3) → P4(priority 4) (*P1 and P3 tie at priority 3; P1 comes first in arrival order.*)

Gantt Chart:

P2	P5	P1	P3	P4
0 1		6	16 18	19

Calculations:

Process	Completion	Turnaround	Waiting
P2	1	1	0
P5	6	6	1
P1	16	16	6
P3	18	18	16
P4	19	19	18

- **Average Waiting Time** = $(0 + 1 + 6 + 16 + 18) / 5 = 41 / 5 = 8.2 \text{ ms}$
- **Average Turnaround Time** = $(1 + 6 + 16 + 18 + 19) / 5 = 60 / 5 = 12.0 \text{ ms}$

Algorithm 4: Round Robin (RR) with Quantum = 1

Rule: Each process gets 1 ms of CPU time in turn, cycling through the ready queue.

Execution Trace:

Time	Running	Remaining (P1,P2,P3,P4,P5)
0-1	P1	9,1,2,1,5
1-2	P2	9,0,2,1,5 → P2 done at t=2
2-3	P3	9,-,1,1,5
3-4	P4	9,-,1,0,5 → P4 done at t=4
4-5	P5	9,-,1,-,4
5-6	P1	8,-,1,-,4
6-7	P3	8,-,0,-,4 → P3 done at t=7
7-8	P5	8,-,-,-,3
8-9	P1	7,-,-,-,3
9-10	P5	7,-,-,-,2
10-11	P1	6,-,-,-,2
11-12	P5	6,-,-,-,1
12-13	P1	5,-,-,-,1

Time	Running	Remaining (P1,P2,P3,P4,P5)
13-14	P5	5,-,-,0 → P5 done at t=14
14-19	P1	0 → P1 done at t=19

Gantt Chart:

```

|P1|P2|P3|P4|P5|P1|P3|P5|P1|P5|P1|P5|P1|P5|P1|P1|P1|P1|P1|
0  1  2  3  4  5  6  7  8  9 10 11 12 13 14                19

```

Calculations:

Process	Completion	Turnaround	Waiting (TAT-Burst)
P1	19	19	9
P2	2	2	1
P3	7	7	5
P4	4	4	3
P5	14	14	9

- **Average Waiting Time** = $(9 + 1 + 5 + 3 + 9) / 5 = 27 / 5 = 5.4 \text{ ms}$
- **Average Turnaround Time** = $(19 + 2 + 7 + 4 + 14) / 5 = 46 / 5 = 9.2 \text{ ms}$

Final Summary Table

Algorithm	Avg Waiting Time	Avg Turnaround Time
FCFS	9.6 ms	13.4 ms
SJF	3.2 ms ← Minimum	7.0 ms
Non-Preemptive Priority	8.2 ms	12.0 ms
Round Robin (q=1)	5.4 ms	9.2 ms

Conclusion: SJF gives the minimum average waiting time. However, SJF requires knowledge of future burst times, which is not always possible in practice.

Q14. FCFS vs SJF Turnaround Time + Idle-First SJF

★★★ [16-17 FINAL] [18-19 FINAL] [19-20 FINAL] — *Book Exercise 5.3*

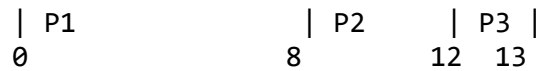
Given Data (Book Exercise 5.3)

Process	Burst Time	Arrival Time
P1	8 ms	0.0 ms
P2	4 ms	0.4 ms
P3	1 ms	1.0 ms

Part A: FCFS (Nonpreemptive)

Rule: Serve in order of arrival. P1 arrives first at $t=0$.

Gantt Chart:



(P2 and P3 arrive before P1 finishes, so they wait.)

Calculations:

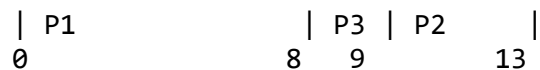
Process	Start	Finish	Turnaround (Finish - Arrival)
P1	0	8	$8 - 0.0 = \mathbf{8.0\ ms}$
P2	8	12	$12 - 0.4 = \mathbf{11.6\ ms}$
P3	12	13	$13 - 1.0 = \mathbf{12.0\ ms}$

- **Average Turnaround Time (FCFS)** = $(8.0 + 11.6 + 12.0) / 3 = 31.6 / 3 = \approx \mathbf{10.53\ ms}$

Part B: SJF (Nonpreemptive)

Rule: At $t=0$, only P1 is available $\rightarrow$ P1 must run. At $t=8$, both P2 and P3 are available $\rightarrow$ choose shortest burst = P3(1 ms).

Gantt Chart:



Calculations:

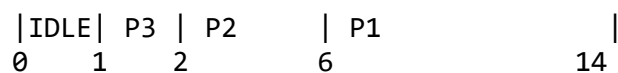
Process	Start	Finish	Turnaround (Finish - Arrival)
P1	0	8	$8 - 0.0 = \mathbf{8.0\ ms}$
P3	8	9	$9 - 1.0 = \mathbf{8.0\ ms}$
P2	9	13	$13 - 0.4 = \mathbf{12.6\ ms}$

- **Average Turnaround Time (SJF)** = $(8.0 + 8.0 + 12.6) / 3 = 28.6 / 3 = \approx \mathbf{9.53\ ms}$

Part C: Idle-First SJF (CPU idle for 1 unit, then SJF)

Rule: CPU is deliberately left **idle from $t=0$ to $t=1$** . At $t=1$, all three processes are available: P1(8), P2(4), P3(1). SJF selects shortest: P3(1) $\rightarrow$ P2(4) $\rightarrow$ P1(8).

Gantt Chart:



Calculations:

Process	Start	Finish	Turnaround (Finish – Arrival)
P3	1	2	$2 - 1.0 = \mathbf{1.0\ ms}$
P2	2	6	$6 - 0.4 = \mathbf{5.6\ ms}$
P1	6	14	$14 - 0.0 = \mathbf{14.0\ ms}$

- **Average Turnaround Time (Idle SJF)** = $(1.0 + 5.6 + 14.0) / 3 = 20.6 / 3 = \approx \mathbf{6.87\ ms}$

Final Comparison

Algorithm	Avg Turnaround Time
FCFS	10.53 ms
SJF (no idle)	9.53 ms

Idle 1 unit + SJF 6.87 ms ← Best

Conclusion: The idle-first SJF has the best average turnaround time. By waiting 1 ms for all processes to arrive, the scheduler can optimally apply SJF to all three processes — demonstrating that **sometimes doing nothing first leads to better overall performance** (future-knowledge scheduling).

Q15. Preemptive Priority + Round Robin Scheduling

★★★ [17-18 FINAL] [20-21 FINAL] — *Book Exercise 5.5, 5.18*

Given Data (Book Exercise 5.5, Time Quantum = 10 ms)

Higher priority number = Higher priority. Preemptive.

Process	Priority	Burst Time	Arrival Time
P1	40	20 ms	0 ms
P2	30	25 ms	25 ms
P3	30	25 ms	30 ms
P4	35	15 ms	60 ms
P5	5	10 ms	100 ms
P6	10	10 ms	105 ms

Scheduling Rules

1. **Preemptive by priority:** A higher-priority process arriving preempts the running process.
2. **RR within same priority:** Processes with equal priority share CPU via Round Robin with quantum = 10 ms.

Execution Trace

t=0: P1 arrives (only process, priority 40). P1 starts. **t=20:** P1 completes. CPU idle (P2 arrives at t=25).

t=25: P2 arrives (priority 30). P2 starts. **t=30:** P3 arrives (priority 30 = same as P2). P2 continues until its current quantum expires. - P2 started at t=25 → quantum expires at t=35.

t=35: P2's quantum done (remaining = 25-10 = 15). P3 starts (RR). **t=45:** P3's quantum done (remaining = 25-10 = 15). P2 starts (RR). **t=55:** P2's quantum done (remaining = 15-10 = 5). P3 starts.

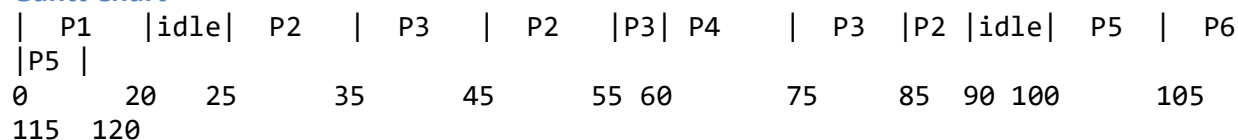
t=60: P4 arrives (priority 35 > 30). **P3 is preempted** (P3 ran for 5 ms, remaining = 10 ms). P4 starts.

t=75: P4 completes. P3 resumes (remaining = 10 ms). **t=85:** P3 completes. P2 starts (remaining = 5 ms). **t=90:** P2 completes. CPU idle (P5 arrives at t=100).

t=100: P5 arrives (priority 5). P5 starts. **t=105:** P6 arrives (priority 10 > 5). **P5 is preempted** (ran 5 ms, remaining = 5 ms). P6 starts.

t=115: P6 completes. P5 resumes (remaining = 5 ms). **t=120:** P5 completes.

Gantt Chart



(Tick marks at: 0, 20, 25, 35, 45, 55, 60, 75, 85, 90, 100, 105, 115, 120)

Calculations

Process	Completion	Turnaround (CT-AT)	Burst	Waiting (TAT-Burst)
P1	20	20-0 = 20	20	0
P2	90	90-25 = 65	25	40
P3	85	85-30 = 55	25	30
P4	75	75-60 = 15	15	0
P5	120	120-100 = 20	10	10
P6	115	115-105 = 10	10	0

- **Average Turnaround Time** = (20+65+55+15+20+10) / 6 = 185/6 ≈ **30.83 ms**

- **Average Waiting Time** = $(0+40+30+0+10+0) / 6 = 80/6 \approx 13.33 \text{ ms}$

Key Observations

1. **P4** (priority 35) immediately preempted P3 when it arrived at $t=60$ — demonstrating preemptive behavior.
 2. **P6** (priority 10) preempted **P5** (priority 5) at $t=105$ — even though P5 arrived first.
 3. **P2 and P3** shared the CPU using Round Robin since they have the same priority (30).
 4. **CPU was idle** from $t=20$ to $t=25$ (no process available) and from $t=90$ to $t=100$.
-
-

QUICK REFERENCE SUMMARY

★★★Questions by Chapter

Chapter	Key ★★★ Topics
Ch. 1	Three purposes of OS; “wasting” resources justification
Ch. 2	Purpose of system calls; parameter-passing methods; OS design strategies
Ch. 3	Process definition and parts; process states; shared-memory vs message-passing
Ch. 4	Thread definition and components; multithreading benefits/challenges
Ch. 5	Preemptive vs nonpreemptive; scheduling criteria; FCFS/SJF/Priority/RR problems

Important Formulas

Turnaround Time (TAT) = Completion Time - Arrival Time
 Waiting Time (WT) = Turnaround Time - Burst Time
 Response Time = First CPU Start - Arrival Time
 Average WT/TAT = Sum of all WT or TAT / Number of Processes

Amdahl's Law: $\text{Speedup} = 1 / [S + (1-S)/N]$
 where S = serial fraction, N = number of cores

EAT (TLB): $\text{EAT} = \alpha \times (\text{TLB access} + \text{Memory access})$
 $\quad + (1-\alpha) \times (\text{TLB access} + 2 \times \text{Memory access})$
 where α = TLB hit ratio

End of ★★ ★Answers — Mahbub Sir's Part (Chapters 1–5) Reference: Operating System Concepts, 10th Edition — Silberschatz, Galvin & Gagne